

TEST FOR ANYLOGISTIX API

The API info is here: <https://anylogistix.help/api/python/readme.html>

PHASE 1 CONNECTION AND SIMULATION WITH THE ANYLOGISTIX API

Install the dependencies

It assumes that the openapi_client is in the local directory ./openapi-python-client-3.3.1

```
In [1]: import sys
import os
# This is my personal directory
os.chdir(r"C:\Users\Luis\Downloads\TFG\API")
#!{sys.executable} -m pip install "pydantic>=2.10,<3"
!{sys.executable} -m pip install "pydantic<2,>=1.10.5"
!{sys.executable} -m pip install -e ./openapi-python-client-3.3.1
```

```

Requirement already satisfied: pydantic<2,>=1.10.5 in c:\users\luis\downloads\tfg\anaconda\envs\tfg_anylogistix\lib\site-packages (1.10.26)
Requirement already satisfied: typing-extensions>=4.2.0 in c:\users\luis\downloads\tfg\anaconda\envs\tfg_anylogistix\lib\site-packages (from pydantic<2,>=1.10.5) (4.15.0)
Obtaining file:///C:/Users/Luis/Downloads/TFG/API/openapi-python-client-3.3.1
  Installing build dependencies: started
  Installing build dependencies: finished with status 'done'
  Checking if build backend supports build_editable: started
  Checking if build backend supports build_editable: finished with status 'done'
  Getting requirements to build editable: started
  Getting requirements to build editable: finished with status 'done'
  Preparing editable metadata (pyproject.toml): started
  Preparing editable metadata (pyproject.toml): finished with status 'done'
Requirement already satisfied: urllib3<2.1.0,>=1.25.3 in c:\users\luis\downloads\tfg\anaconda\envs\tfg_anylogistix\lib\site-packages (from openapi-client==1.0.0) (2.0.7)
Requirement already satisfied: python-dateutil in c:\users\luis\downloads\tfg\anaconda\envs\tfg_anylogistix\lib\site-packages (from openapi-client==1.0.0) (2.9.0.post0)
Requirement already satisfied: pydantic<2,>=1.10.5 in c:\users\luis\downloads\tfg\anaconda\envs\tfg_anylogistix\lib\site-packages (from openapi-client==1.0.0) (1.10.26)
Requirement already satisfied: aenum in c:\users\luis\downloads\tfg\anaconda\envs\tfg_anylogistix\lib\site-packages (from openapi-client==1.0.0) (3.1.17)
Requirement already satisfied: typing-extensions>=4.2.0 in c:\users\luis\downloads\tfg\anaconda\envs\tfg_anylogistix\lib\site-packages (from pydantic<2,>=1.10.5->openapi-client==1.0.0) (4.15.0)
Requirement already satisfied: six>=1.5 in c:\users\luis\downloads\tfg\anaconda\envs\tfg_anylogistix\lib\site-packages (from python-dateutil->openapi-client==1.0.0) (1.17.0)
Building wheels for collected packages: openapi-client
  Building editable for openapi-client (pyproject.toml): started
  Building editable for openapi-client (pyproject.toml): finished with status 'done'
  Created wheel for openapi-client: filename=openapi_client-1.0.0-0.editable-py3-none-any.whl size=3042 sha256=7b9a704f83785e1511032fe20a89bf6436de699e2ebee877b070a6917eae171c
  Stored in directory: C:\Users\Luis\AppData\Local\Temp\pip-ephem-wheel-cache-hln00r8c\wheels\96\67\f9\218cef519d1de93e10ecbd2f12fb5f4d1d0e81b1c1d5296ba2
Successfully built openapi-client
Installing collected packages: openapi-client
  Attempting uninstall: openapi-client
    Found existing installation: openapi-client 1.0.0
    Uninstalling openapi-client-1.0.0:
      Successfully uninstalled openapi-client-1.0.0
Successfully installed openapi-client-1.0.0

```

```

In [7]: import openapi_client
        from openapi_client.rest import ApiException
        from pprint import pprint
        import urllib3

```

```

In [8]: # SERVER_IP="192.168.64.159"
        # SERVER_IP="192.168.67.110"
        SERVER_IP="alxserver.aut.uah.es"
        SERVER_URL=f"https://{SERVER_IP}:443/api/v1"

        # This is my personal APY KEY
        API_KEY="c184f1ab-9f13-484c-a1c1-3d543502da6e"

```

```
SERVER_URL
```

```
Out[8]: 'https://alxserver.aut.uah.es:443/api/v1'
```

PART I: CHECK CONNECTIVITY AND OPERATIONAL STATUS

```
In [9]: # Defining the host is optional and defaults to https://server_address:port/api/
# See configuration.py for a list of all supported configuration parameters.
configuration = openapi_client.Configuration(
    host = SERVER_URL
)
# The client must configure the authentication and authorization parameters
# in accordance with the API server security policy.
# Examples for each auth method are provided below, use the example that
# satisfies your auth use case.
# Configure API key authorization: ApiKey
configuration.api_key['ApiKey'] = API_KEY

# ALVARO: SUPER IMPORTANTE PARA PODER TRABAJAR CON CERTIFICADOS AUTO-FIRMADOS
configuration.verify_ssl = False
urllib3.disable_warnings(urllib3.exceptions.InsecureRequestWarning)
with openapi_client.ApiClient(configuration) as api_client:
    # Create an instance of the API class
    api_instance = openapi_client.OpenApiApi(api_client)
```

Obtain current user data

```
In [10]: try:
# Gets current user data.
api_response = api_instance.get_current_user()
print("The response of OpenApiApi->get_current_user:\n")
pprint(api_response)
except Exception as e:
print("Exception when calling OpenApiApi->get_current_user: %s\n" % e)
```

The response of OpenApiApi->get_current_user:

```
ApiUserData(email='adrian.encabo@edu.uah.es', first_name='adrian.encabo@edu.uah.es', id=403, last_name='', username='adrian.encabo@edu.uah.es')
```

PART II: OBTAIN PROJECTS, SCENARIOS, EXECUTIONS

Obtain the list of currently available projects

```
In [11]: try:
# Returns a list of projects that the user has access to.
api_response = api_instance.get_projects()
print("The response of OpenApiApi->get_projects:\n")
pprint(api_response)
except Exception as e:
print("Exception when calling OpenApiApi->get_projects: %s\n" % e)
```

The response of OpenApiApi->get_projects:

```
[ApiProjectResponse(accessible=True, creation_date='2026-03-10T16:19:55.145394Z',
current_user_id=403, deleted=False, id=79020, name='TFG_ADRIAN_ENCABO', owner_user_id=3)]
```

Open a project and get the list of scenarios available

```
In [12]: # My personal Project defined
MY_PROJECT_NAME="TFG_ADRIAN_ENCABO"
the_project_id=0
try:
    full_match = True # bool | Whether to match project name by complete match.
    # Opens project with name projectName.
    the_project = api_instance.find_and_open_project_by_name(full_match, MY_PROJECT_NAME)
    print("The response of OpenApiApi->find_and_open_project_by_name:\n")
    pprint(the_project)
except Exception as e:
    print("Exception when calling OpenApiApi->find_and_open_project_by_name: %s\n" % e)
try:
    # Returns a List of scenarios for the project with identifier project_id.
    the_scenarios = api_instance.get_scenarios(the_project.id)
    print("The response of OpenApiApi->get_scenarios:\n")
    pprint(the_scenarios)
except Exception as e:
    print("Exception when calling OpenApiApi->get_scenarios: %s\n" % e)
```

The response of OpenApiApi->find_and_open_project_by_name:

```
ApiProjectResponse(accessible=True, creation_date='2026-03-10T16:19:55.145394Z',
current_user_id=403, deleted=False, id=79020, name='TFG_ADRIAN_ENCABO', owner_user_id=3)
```

The response of OpenApiApi->get_scenarios:

```
[ApiScenarioData(id=82840, name='Budget Comparison (Estimated Demand)', type='SIM'),
ApiScenarioData(id=87083, name='P3.3. GFA 1. Results 2_Different Locations', type='SIM'),
ApiScenarioData(id=84191, name='Budget Comparison (Estimated Demand) 2', type='SIM'),
ApiScenarioData(id=85538, name='Budget Comparison (Estimated Demand) 2', type='SIM'),
ApiScenarioData(id=86671, name='P4.2.2 NO Distribution Network inside 4 Walls Models Capacity Restriction', type='NO'),
ApiScenarioData(id=88185, name='P4.2.2 NO Distribution Network inside 4 Walls Models Capacity Restriction 2', type='NO'),
ApiScenarioData(id=89212, name='P4.2.2 NO Distribution Network inside 4 Walls Models Capacity Restriction sim', type='SIM'),
ApiScenarioData(id=92735, name='P4.2.6 SIM Distribution Network-4 Walls_New Inventory+New Transportation', type='SIM'),
ApiScenarioData(id=95044, name='null', type='SIM'),
ApiScenarioData(id=176431, name='Cold Chain (Risk Analysis)', type='SIM'),
ApiScenarioData(id=177767, name='US Distribution Network', type='NO'),
ApiScenarioData(id=178297, name='Lumber Production', type='NO'),
ApiScenarioData(id=179567, name='Global Network Examination', type='SIM'),
ApiScenarioData(id=180669, name='asset-v1_UAH+EUGLOH+EUG2_T2_1_0080+type@asset+block@P4.4_Demand_Variation_LoadingGates_2', type='SIM'),
ApiScenarioData(id=181707, name='P3.1._Green_Field_Analysis_GFA__ALX3', type='GFA')]
```

Select the scenario by name

```
In [13]: #Add the name of the scenario
MY_SCENARIO_NAME="P4.2.6 SIM Distribution Network-4 Walls_New Inventory+New Tran

# 1. Select the scenario
the_scenario = next(s for s in the_scenarios if s.name == MY_SCENARIO_NAME)
pprint(the_scenario)
```

ApiScenarioData(id=92735, name='P4.2.6 SIM Distribution Network-4 Walls_New Inventory+New Transportation', type='SIM')

Get the list of experiment runs of a scenario

```
In [14]: try:
# Returns the results of experiment runs for scenario.
the_runs = api_instance.get_experiment_runs_for_scenario(the_scenario.id)
print("The response of OpenApiApi->get_experiment_runs_for_scenario:\n")
pprint(the_runs)
except Exception as e:
print("Exception when calling OpenApiApi->get_experiment_runs_for_scenario:
```

The response of OpenApiApi->get_experiment_runs_for_scenario:

```
[ApiBasicRun(has_options=False, id=93824, name='Statistics', rc_id=93825, removing=False, scenario_id=92735, type='SIMULATION'),
 ApiBasicRun(has_options=False, id=94035, name='Statistics 2', rc_id=94036, removing=False, scenario_id=92735, type='SIMULATION'),
 ApiBasicRun(has_options=False, id=94246, name='Statistics 3', rc_id=94247, removing=False, scenario_id=92735, type='SIMULATION'),
 ApiBasicRun(has_options=False, id=94433, name='Statistics 4', rc_id=94434, removing=False, scenario_id=92735, type='SIMULATION'),
 ApiBasicRun(has_options=False, id=94629, name='Statistics 5', rc_id=94630, removing=False, scenario_id=92735, type='SIMULATION'),
 ApiBasicRun(has_options=False, id=94806, name='Statistics 6', rc_id=94807, removing=False, scenario_id=92735, type='SIMULATION')]
```

PART III. EXECUTE A SIMULATION

Get the list of RunConfigurations for the experiments of a given Scenario

Each type of experiment has a RunConfiguration. For example:

```
ApiBasicRunConfiguration(animation_model_id=None,
feasible_check_status='NOT_CONDUCTED', id=2439,
is_feasible_check_data_available=False, model_time=None,
name='i18n-Experiment_Simulation_text', progress=None,
project_id=63, routes_progress=None, run_status=None,
scenario_id=1752, speed=None, status='IDLE', step='IDLE',
type='SIMULATION', validation_status='VALID')
```

Types of available experiment runs are:

- SIMULATION
- VARIATION
- COMPARISON
- SAFETY_STOCK
- RISK_ANALYSIS

```
In [15]: MY_RUN_TYPE="SIMULATION"
try:
    # Returns a list of experiments available for a given scenario.
    the_run_configurations= api_instance.get_experiments(the_project.id, the_sce
    print("The response of OpenApiApi->get_experiments:\n")
    # Exception fixed for undefined variable
    pprint(the_run_configurations)
except Exception as e:
    print("Exception when calling OpenApiApi->get_experiments: %s\n" % e)

# 2. Select the adequate RunConfiguration for the scenario
the_RC = next((r for r in the_run_configurations if r.type == MY_RUN_TYPE), None)
pprint(the_RC)
```

The response of OpenApiApi->get_experiments:

```
[ApiBasicRunConfiguration(animation_model_id=None, feasible_check_status='NOT_CON
DUCTED', id=93422, is_feasible_check_data_available=False, model_time=None, name
='i18n-Experiment_Simulation_text', progress=None, project_id=79020, routes_progr
ess=None, run_status=None, scenario_id=92735, speed=None, status='IDLE', step='ID
LE', type='SIMULATION', validation_status='NOT_CONDUCTED'),
 ApiBasicRunConfiguration(animation_model_id=None, feasible_check_status='NOT_CON
DUCTED', id=93579, is_feasible_check_data_available=False, model_time=None, name
='i18n-Experiment_VariationExperiment_text', progress=None, project_id=79020, rou
tes_progress=None, run_status=None, scenario_id=92735, speed=None, status='IDLE',
step='IDLE', type='VARIATION', validation_status='NOT_CONDUCTED'),
 ApiBasicRunConfiguration(animation_model_id=None, feasible_check_status='NOT_CON
DUCTED', id=92743, is_feasible_check_data_available=False, model_time=None, name
='i18n-Experiment_ComparisonExperiment_text', progress=None, project_id=79020, ro
utes_progress=None, run_status=None, scenario_id=92735, speed=None, status='IDL
E', step='IDLE', type='COMPARISON', validation_status='NOT_CONDUCTED'),
 ApiBasicRunConfiguration(animation_model_id=None, feasible_check_status='NOT_CON
DUCTED', id=93283, is_feasible_check_data_available=False, model_time=None, name
='i18n-Experiment_SafetyStockEstimation_text', progress=None, project_id=79020, r
outes_progress=None, run_status=None, scenario_id=92735, speed=None, status='IDL
E', step='IDLE', type='SAFETY_STOCK', validation_status='NOT_CONDUCTED'),
 ApiBasicRunConfiguration(animation_model_id=None, feasible_check_status='NOT_CON
DUCTED', id=93098, is_feasible_check_data_available=False, model_time=None, name
='i18n-Experiment_RiskAnalysisExperiment_text', progress=None, project_id=79020,
routes_progress=None, run_status=None, scenario_id=92735, speed=None, status='IDL
E', step='IDLE', type='RISK_ANALYSIS', validation_status='NOT_CONDUCTED')]
ApiBasicRunConfiguration(animation_model_id=None, feasible_check_status='NOT_COND
UCTED', id=93422, is_feasible_check_data_available=False, model_time=None, name
='i18n-Experiment_Simulation_text', progress=None, project_id=79020, routes_progr
ess=None, run_status=None, scenario_id=92735, speed=None, status='IDLE', step='ID
LE', type='SIMULATION', validation_status='NOT_CONDUCTED')
```

Run a new experiment over a scenario (synch)

Runs synchronously.

```
In [16]: try:
# Starts the experiment synchronously.
the_sync_result = api_instance.run_experiment_synchronously(the_RC.id)
print("The response of OpenApiApi->run_experiment_synchronously:\n")
pprint(the_sync_result)
except Exception as e:
print("Exception when calling OpenApiApi->run_experiment_synchronously: %s\n"
```

The response of OpenApiApi->run_experiment_synchronously:

```
ApiExperimentResult(experiment_result_id=181988, validation_errors=None, validation_status='OK', validation_warnings=None)
```

Part IIIb. Async execution

Run a new experiment over a scenario (async)

Runs asynchronously. Periodic status check.

```
In [17]: skip_scenario_warnings = True # bool / Whether to skip scenario warnings. (optional)
try:
# Starts the experiment asynchronously.
the_execution = api_instance.run_experiment(the_RC.id, skip_scenario_warning
print("The response of OpenApiApi->run_experiment:\n")
pprint(the_execution)
except Exception as e:
print("Exception when calling OpenApiApi->run_experiment: %s\n" % e)

import time
while True:
try:
# Returns the experiment status
the_status = api_instance.get_experiment_status(the_RC.id)
# pprint(api_response)

# Check if execution finished
if the_status.finished:
print("Execution finished.")
break

# Optional: stop if failed
# fixed exception of status check
if the_status.failed:
print("Execution failed.")
break

except Exception as e:
print("Exception when calling OpenApiApi->get_experiment_status:", e)
print("Not finished yet. Wait 1sec and check again")
# wait before next check
time.sleep(1)
```

The response of OpenApiApi->run_experiment:

```
ApiRunExperiment(animation_model_id=93422, experiment_type_id='SIMULATION', is3d_animation=False)
Execution finished.
```

Get the experiments results (pages)

You can ask for limited number of results (avoid overflow)

```
In [18]: skip = 0 # int / Number of records to skip. (default to 0)
         take = 1000 # int / Number of records to be retrieved. (default to 1000)

         try:
             # Returns all the results of the specific experiment run.
             the_run_results = api_instance.get_experiment_run_result(the_RC.id, skip, ta
             print("The response of OpenApiApi->get_experiment_run_result:\n")
             pprint(the_run_results)
         except Exception as e:
             print("Exception when calling OpenApiApi->get_experiment_run_result: %s\n" %
```

The response of OpenApiApi->get_experiment_run_result:

```
ApiExperimentResultData(pages=[ApiExperimentResultPageWrapper(charts=[ApiExperimentResultGraphChartWrapper(chart=ApiChartMetadataShort(id=93425, is_group_mode_enabled=False, layout=ApiChartLayoutData(height=1, start_col=0, start_row=0, width=2), name='Transportation Cost, Revenue, Total Cost, Profit, Inventory Carrying Cost', rc_id=93422, type='BAR'), class_name='ApiExperimentResultGraphChartWrapper', data=ApiGraphChartData(entity_list=[], id=93425, show_x=True, total_entity_count=0, total_technical_row_count=0, type='BAR')), ApiExperimentResultGraphChartWrapper(chart=ApiChartMetadataShort(id=93426, is_group_mode_enabled=False, layout=ApiChartLayoutData(height=1, start_col=2, start_row=0, width=2), name='ELT Service Level by Products', rc_id=93422, type='LINE'), class_name='ApiExperimentResultGraphChartWrapper', data=ApiGraphChartData(entity_list=[], id=93426, show_x=True, total_entity_count=0, total_technical_row_count=0, type='LINE')), ApiExperimentResultGraphChartWrapper(chart=ApiChartMetadataShort(id=93427, is_group_mode_enabled=False, layout=ApiChartLayoutData(height=1, start_col=4, start_row=0, width=2), name='Lead time', rc_id=93422, type='LINE'), class_name='ApiExperimentResultGraphChartWrapper', data=ApiGraphChartData(entity_list=[], id=93427, show_x=True, total_entity_count=0, total_technical_row_count=0, type='LINE')), ApiExperimentResultGraphChartWrapper(chart=ApiChartMetadataShort(id=93428, is_group_mode_enabled=False, layout=ApiChartLayoutData(height=1, start_col=0, start_row=1, width=2), name='Lead time', rc_id=93422, type='HISTOGRAM'), class_name='ApiExperimentResultGraphChartWrapper', data=ApiGraphChartData(entity_list=[], id=93428, show_x=True, total_entity_count=0, total_technical_row_count=0, type='HISTOGRAM')), ApiExperimentResultTableChartWrapper(chart=ApiChartMetadataShort(id=93429, is_group_mode_enabled=False, layout=ApiChartLayoutData(height=1, start_col=2, start_row=1, width=2), name='Transportation Cost, Revenue, Total Cost, Profit, Inventory Carrying Cost', rc_id=93422, type='TABLE'), class_name='ApiExperimentResultTableChartWrapper', data=ApiStrictTableChartData(entity_list=[], id=93429, total_entity_count=0, total_technical_row_count=0, type='TABLE')), ApiExperimentResultTableChartWrapper(chart=ApiChartMetadataShort(id=93430, is_group_mode_enabled=False, layout=ApiChartLayoutData(height=1, start_col=4, start_row=1, width=2), name='Demand (Orders Backlog), Fulfillment (Late Orders), Fulfillment Received (Orders On-time), Fulfillment Received (Orders) by Customer', rc_id=93422, type='TABLE'), class_name='ApiExperimentResultTableChartWrapper', data=ApiStrictTableChartData(entity_list=[], id=93430, total_entity_count=0, total_technical_row_count=0, type='TABLE'))], page=ApiPage(id='93424', name='Financial and customer performance')), ApiExperimentResultPageWrapper(charts=[ApiExperimentResultGraphChartWrapper(chart=ApiChartMetadataShort(id=93432, is_group_mode_enabled=False, layout=ApiChartLayoutData(height=1, start_col=0, start_row=0, width=2), name='Peak Capacity', rc_id=93422, type='HISTOGRAM'), class_name='ApiExperimentResultGraphChartWrapper', data=ApiGraphChartData(entity_list=[], id=93432, show_x=True, total_entity_count=0, total_technical_row_count=0, type='HISTOGRAM')), ApiExperimentResultGraphChartWrapper(chart=ApiChartMetadataShort(id=93433, is_group_mode_enabled=False, layout=ApiChartLayoutData(height=1, start_col=2, start_row=0, width=2), name='Peak Capacity', rc_id=93422, type='LINE'), class_name='ApiExperimentResultGraphChartWrapper', data=ApiGraphChartData(entity_list=[], id=93433, show_x=True, total_entity_count=0, total_technical_row_count=0, type='LINE')), ApiExperimentResultGraphChartWrapper(chart=ApiChartMetadataShort(id=93434, is_group_mode_enabled=False, layout=ApiChartLayoutData(height=1, start_col=4, start_row=0, width=2), name='Available Inventory', rc_id=93422, type='LINE'), class_name='ApiExperimentResultGraphChartWrapper', data=ApiGraphChartData(entity_list=[], id=93434, show_x=True, total_entity_count=0, total_technical_row_count=0, type='LINE')), ApiExperimentResultGraphChartWrapper(chart=ApiChartMetadataShort(id=93435, is_group_mode_enabled=False, layout=ApiChartLayoutData(height=1, start_col=0, start_row=1, width=2), name='Available Inventory in Product Units', rc_id=93422, type='LINE'), class_name='ApiExperimentResultGraphChartWrapper', data=ApiGraphChartData(entity_list=[], id=93435, show_x=True, total_entity_count=0, total_technical_row_count=0, type='LINE')), ApiExperimentResultGraphChartWrapper(chart=ApiChartMetadataShort(id=93436, is_group_mode_enabled=False, layout=ApiChartLayoutData(height=1, start_col=2, start_row=1, width=2), name='Available
```

```
Inventory in Product Units', rc_id=93422, type='LINE')), class_name='ApiExperiment
ResultGraphChartWrapper', data=ApiGraphChartData(entity_list=[], id=93436, show_x
=True, total_entity_count=0, total_technical_row_count=0, type='LINE')), ApiExper
imentResultGraphChartWrapper(chart=ApiChartMetadataShort(id=93437, is_group_mode_
enabled=False, layout=ApiChartLayoutData(height=1, start_col=4, start_row=1, widt
h=2), name='Available Inventory in Product Units', rc_id=93422, type='HISTOGRAM'
)), class_name='ApiExperimentResultGraphChartWrapper', data=ApiGraphChartData(en
tity_list=[], id=93437, show_x=True, total_entity_count=0, total_technical_row_co
unt=0, type='HISTOGRAM'))], page=ApiPage(id='93431', name='Operational performanc
e ')), ApiExperimentResultPageWrapper(charts=[ApiExperimentResultGraphChartWrape
r(chart=ApiChartMetadataShort(id=93439, is_group_mode_enabled=False, layout=ApiCh
artLayoutData(height=1, start_col=0, start_row=0, width=2), name='Available Inven
tory in Product Units', rc_id=93422, type='LINE')), class_name='ApiExperimentResul
tGraphChartWrapper', data=ApiGraphChartData(entity_list=[], id=93439, show_x=Tru
e, total_entity_count=0, total_technical_row_count=0, type='LINE')), ApiExperimen
tResultGraphChartWrapper(chart=ApiChartMetadataShort(id=93441, is_group_mode_enab
led=False, layout=ApiChartLayoutData(height=1, start_col=2, start_row=0, width=
2), name='Peak Capacity', rc_id=93422, type='HISTOGRAM')), class_name='ApiExperi
mentResultGraphChartWrapper', data=ApiGraphChartData(entity_list=[], id=93441, sh
ow_x=True, total_entity_count=0, total_technical_row_count=0, type='HISTOGRAM')), A
piExperimentResultGraphChartWrapper(chart=ApiChartMetadataShort(id=93442, is_grou
p_mode_enabled=False, layout=ApiChartLayoutData(height=1, start_col=4, start_row=
0, width=2), name='Peak Capacity', rc_id=93422, type='LINE')), class_name='ApiExpe
rimentResultGraphChartWrapper', data=ApiGraphChartData(entity_list=[], id=93442,
show_x=True, total_entity_count=0, total_technical_row_count=0, type='LINE')), Ap
iExperimentResultGraphChartWrapper(chart=ApiChartMetadataShort(id=93443, is_group
_mode_enabled=False, layout=ApiChartLayoutData(height=1, start_col=0, start_row=
1, width=2), name='Available Inventory in Product Units', rc_id=93422, type='LIN
E')), class_name='ApiExperimentResultGraphChartWrapper', data=ApiGraphChartData(en
tity_list=[], id=93443, show_x=True, total_entity_count=0, total_technical_row_co
unt=0, type='LINE')), ApiExperimentResultGraphChartWrapper(chart=ApiChartMetadata
Short(id=93445, is_group_mode_enabled=False, layout=ApiChartLayoutData(height=1,
start_col=2, start_row=1, width=2), name='Available Inventory in Product Units',
rc_id=93422, type='LINE')), class_name='ApiExperimentResultGraphChartWrapper', dat
a=ApiGraphChartData(entity_list=[], id=93445, show_x=True, total_entity_count=0,
total_technical_row_count=0, type='LINE'))], page=ApiPage(id='93438', name='Inven
tory and capacity dynamics')))]
```

```
In [19]: for p in the_run_results.pages:
          print(f"\n{p.page.name} (id:{p.page.id})")
          for c in p.charts:
              print(f"    - {c.chart.name} (type: {c.chart.type}) (id: {c.chart.id})")
```

Financial and customer performance (id:93424)

- Transportation Cost, Revenue, Total Cost, Profit, Inventory Carrying Cost (type: BAR) (id: 93425)
- ELT Service Level by Products (type: LINE) (id: 93426)
- Lead time (type: LINE) (id: 93427)
- Lead time (type: HISTOGRAM) (id: 93428)
- Transportation Cost, Revenue, Total Cost, Profit, Inventory Carrying Cost (type: TABLE) (id: 93429)
- Demand (Orders Backlog), Fulfillment (Late Orders), Fulfillment Received (Orders On-time), Fulfillment Received (Orders) by Customer (type: TABLE) (id: 93430)

Operational performance (id:93431)

- Peak Capacity (type: HISTOGRAM) (id: 93432)
- Peak Capacity (type: LINE) (id: 93433)
- Available Inventory (type: LINE) (id: 93434)
- Available Inventory in Product Units (type: LINE) (id: 93435)
- Available Inventory in Product Units (type: LINE) (id: 93436)
- Available Inventory in Product Units (type: HISTOGRAM) (id: 93437)

Inventory and capacity dynamics (id:93438)

- Available Inventory in Product Units (type: LINE) (id: 93439)
- Peak Capacity (type: HISTOGRAM) (id: 93441)
- Peak Capacity (type: LINE) (id: 93442)
- Available Inventory in Product Units (type: LINE) (id: 93443)
- Available Inventory in Product Units (type: LINE) (id: 93445)

Obtain the pages in the experiment's result dashboard

```
In [40]: try:
    # Returns statistics pages for the result of experiment run.
    # Exception fixed for 'id' attribute
    # We use the experiment_result_id from the previously executed synchronous r
    result_id = the_sync_result.experiment_result_id
    dashboard_pages = api_instance.get_experiment_dashboard_pages(result_id)
    print("The response of OpenApiApi->get_experiment_dashboard_pages:\n")

    # CHANGED Displaying the available pages and charts with their dynamic IDs
    for page in dashboard_pages:
        print(f"Dashboard Page: {page.name} (ID: {page.id})")
        for chart in page.charts:
            print(f" - Chart: {chart.name} (ID: {chart.id})")

except Exception as e:
    print("Exception when calling OpenApiApi->get_experiment_dashboard_pages: %s")
```

The response of OpenApiApi->get_experiment_dashboard_pages:

Dashboard Page: Financial and customer performance (ID: 181990)

- Chart: Transportation Cost, Revenue, Total Cost, Profit, Inventory Carrying Cost (ID: 181991)
- Chart: ELT Service Level by Products (ID: 181992)
- Chart: Lead time (ID: 181993)
- Chart: Lead time (ID: 181994)
- Chart: Transportation Cost, Revenue, Total Cost, Profit, Inventory Carrying Cost (ID: 181995)
- Chart: Demand (Orders Backlog), Fulfillment (Late Orders), Fulfillment Received (Orders On-time), Fulfillment Received (Orders) by Customer (ID: 181996)

Dashboard Page: Operational performance (ID: 181997)

- Chart: Peak Capacity (ID: 181998)
- Chart: Peak Capacity (ID: 181999)
- Chart: Available Inventory (ID: 182000)
- Chart: Available Inventory in Product Units (ID: 182001)
- Chart: Available Inventory in Product Units (ID: 182002)
- Chart: Available Inventory in Product Units (ID: 182003)

Dashboard Page: Inventory and capacity dynamics (ID: 182004)

- Chart: Available Inventory in Product Units (ID: 182005)
- Chart: Peak Capacity (ID: 182007)
- Chart: Peak Capacity (ID: 182008)
- Chart: Available Inventory in Product Units (ID: 182009)
- Chart: Available Inventory in Product Units (ID: 182011)

Export the results as an excel matrix

```
In [21]: # Exception fixed for wrong IDs
import shutil #For manipulating excel files
try:
    # Dynamically select the first dashboard page to export to avoid hardcoded ID
    if dashboard_pages and len(dashboard_pages) > 0:
        target_page_id = dashboard_pages[0].id
        target_result_id = the_sync_result.experiment_result_id

        print(f"Exporting dashboard page ID {target_page_id} from result ID {target_result_id}")

        # Returns an Excel representation of the dashboard page
        excel_export = api_instance.export_dashboard_page(target_result_id, target_page_id)

        # Save the exported Excel file locally
        output_filename = "Simulation_Results.xlsx"

        # Check if the API returns a temporary file path
        if isinstance(excel_export, str) and os.path.exists(excel_export):
            shutil.move(excel_export, output_filename)
            print(f"Excel file saved successfully to: {os.path.abspath(output_filename)}")

        # Check if the API returns binary data directly
        elif isinstance(excel_export, bytes):
            with open(output_filename, "wb") as file:
                file.write(excel_export)
            print(f"Excel file saved successfully to: {os.path.abspath(output_filename)}")

        else:
            print(f"Unexpected data format received. Type: {type(excel_export)}")
            pprint(excel_export)
```

```

        print("\nData exported successfully.")
    else:
        print("No dashboard pages available to export.")

except Exception as e:
    print("Exception when calling OpenApiApi->export_dashboard_page: %s\n" % e)

```

Exporting dashboard page ID 181990 from result ID 181988...

Excel file saved successfully to: C:\Users\Luis\Downloads\TFG\API\Simulation_Results.xlsx

Data exported successfully.

Close current project

```

In [41]: #Error fixed of indentation
try:
    # Closes the currently open project.
    api_response = api_instance.close_project()
    print("The response of OpenApiApi->close_project:\n")
    pprint(api_response)
    print("\nProject closed successfully.")
except ApiException as e:
    print("Exception when calling OpenApiApi->close_project: %s\n" % e)

```

The response of OpenApiApi->close_project:

```

ApiProjectResponse(accessible=True, creation_date='2026-03-10T16:19:55.145394Z',
current_user_id=None, deleted=False, id=79020, name='TFG_ADRIAN_ENCABO', owner_user_id=3)

```

Project closed successfully.

=====

PHASE 2



PART I Flat AI Decision & Scenario Cloning

1. Define the Flat AI logic (3 predefined decisions)

```

In [52]: decision_options = [
    "DECISION 0: Increase Demand by 20% (Aggressive Growth)",
    "DECISION 1: Decrease Transport Costs by 15% (Cost Optimization)",
    "DECISION 2: Increase Safety Stock by 10% (Conservative Risk Management)"
]

print("--- FLAT AI: EVALUATING PREDEFINED DECISIONS ---")
for i, decision in enumerate(decision_options):
    print(f"[{i}] {decision}")

# For this 'flat AI' phase, we will simulate the AI choosing a decision automati

```

```
# Let us assume the AI evaluates the current context and selects Decision 0.
selected_decision_index = 0
selected_decision_text = decision_options[selected_decision_index]

print(f"\nFLAT AI SELECTED: {selected_decision_text}")
```

```
--- FLAT AI: EVALUATING PREDEFINED DECISIONS ---
[0] DECISION 0: Increase Demand by 20% (Aggressive Growth)
[1] DECISION 1: Decrease Transport Costs by 15% (Cost Optimization)
[2] DECISION 2: Increase Safety Stock by 10% (Conservative Risk Management)
```

FLAT AI SELECTED: DECISION 0: Increase Demand by 20% (Aggressive Growth)

2. Clone the existing scenario

```
In [53]: # to apply the decision without altering the base scenario
try:
    print("\nCloning the base scenario to apply the decision safely...")

    # The API requires a body parameter for this request, even if empty
    copy_parameters = openapi_client.ApiConvertAndCopyParameters()

    # We copy the scenario synchronously using the same type as the original.
    cloned_scenario = api_instance.copy_scenario_synchronously(
        new_type=the_scenario.type,
        scenario_id=the_scenario.id,
        body=copy_parameters
    )

    print("The response of OpenApiApi->copy_scenario_synchronously:\n")
    pprint(cloned_scenario)

    print("\nScenario successfully cloned.")
    print(f"New Cloned Scenario ID: {cloned_scenario.id}")
    print(f"New Cloned Scenario Name: {cloned_scenario.name}")

    # We store the new scenario in a specific variable to work with it in the ne
    the_cloned_scenario = cloned_scenario

except Exception as e:
    print("Exception when calling OpenApiApi->copy_scenario_synchronously: %s\n"
```

Cloning the base scenario to apply the decision safely...

The response of OpenApiApi->copy_scenario_synchronously:

```
ApiScenarioData(id=192031, name=None, type='SIM')
```

Scenario successfully cloned.

New Cloned Scenario ID: 192031

New Cloned Scenario Name: None

3. Configure the cloned scenario

```
In [54]: try:
    print(f"--- PREPARING CLONED SCENARIO (ID: {the_cloned_scenario.id}) ---")

    # 1. Retrieve the available experiments for the newly cloned scenario
    print("Fetching available run configurations for the cloned scenario...")
    cloned_run_configurations = api_instance.get_experiments(the_project.id, the
```

```

    if cloned_run_configurations:
        # 2. Automatically select the 'SIMULATION' configuration (or the first a
        cloned_RC = next((r for r in cloned_run_configurations if r.type == 'SIM

        print("\nSelected Run Configuration for the cloned scenario successfully
        print(f" - Configuration ID: {cloned_RC.id}")
        print(f" - Configuration Type: {cloned_RC.type}")

        print("\n[SYSTEM] The cloned scenario is now ready to receive data modif

    else:
        print("\nNo run configurations found for the cloned scenario.")
        cloned_RC = None

except Exception as e:
    print("Exception when configuring the cloned scenario: %s\n" % e)

```

--- PREPARING CLONED SCENARIO (ID: 192031) ---

Fetching available run configurations for the cloned scenario...

Selected Run Configuration for the cloned scenario successfully:

- Configuration ID: 192718
- Configuration Type: SIMULATION

[SYSTEM] The cloned scenario is now ready to receive data modifications.

4.a) Modification by Excel

```

In [55]: # This path requires a local excel file. A standard AI agent would use pandas
          # to open it, apply the 20 percent demand increase, and save it.

import os

# Replace this with the actual path to your modified Excel file if you have one
modified_excel_path = r"C:\Users\Luis\Downloads\TFG\Curso_logistica_2025\Materia

print("--- PATH A: EXCEL IMPORT METHOD ---")

try:
    if os.path.exists(modified_excel_path):
        print(f"Uploading modified Excel file: {modified_excel_path}")

        with open(modified_excel_path, "rb") as file_data:
            # We use import_excel_existing to overwrite the cloned scenario tabl
            api_instance.import_excel_existing(
                new_scenario_name="Modified_Scenario",
                project_id=the_project.id,
                scenario_id=the_cloned_scenario.id,
                file=file_data.read(),
                overwrite_existing_tables=True
            )
        print("Scenario data updated successfully via Excel import.")
    else:
        print(f"File '{modified_excel_path}' not found.")
        print("To fully test this path, place an ALX Excel file in your folder a

except Exception as e:
    print("Exception when calling OpenApiApi->import_excel_existing: %s\n" % e)

```

--- PATH A: EXCEL IMPORT METHOD ---

Uploading modified Excel file: C:\Users\Luis\Downloads\TFG\Curso_logistica_2025\Material\Tema 4 - MULTIMODAL TRANSPORT PLANNING\2. Practice\Ch2. Simulation Files\P4.2.6 SIM Distribution Network-4 Walls_New Inventory+New Transportation.xlsx
Exception when calling OpenApiApi->import_excel_existing: 'utf-8' codec can't decode byte 0xff in position 14: invalid start byte

4.b) Modification by Native API variation

```
In [56]: # This path explores setting up an internal ALX variation.
# Step 1 is to ask the server what objects (tables) can be varied natively.

print("--- PATH B: NATIVE API VARIATION METHOD ---")

try:
    print("Querying the server for available variable parameter object types...")

    # We query the server using the cloned scenario ID
    variable_types = api_instance.get_variable_parameter_object_types(scenario_id)

    print("The response of OpenApiApi->get_variable_parameter_object_types:\n")

    # Printing the available tables that can be modified natively
    if variable_types and variable_types.list:
        for option in variable_types.list:
            print(f" - Table Type: {option.name} (ID: {option.id})")
    else:
        print("No variable types found for this scenario.")

except Exception as e:
    print("Exception when calling OpenApiApi->get_variable_parameter_object_type
```

--- PATH B: NATIVE API VARIATION METHOD ---

Querying the server for available variable parameter object types...

The response of OpenApiApi->get_variable_parameter_object_types:

- Table Type: Customers (ID: CustomerData_GFA_NO_SIM_TO_FULL)
- Table Type: Demand (ID: DemandData_SIM_FULL)
- Table Type: Distribution Centers (ID: DCData_SIM_FULL)
- Table Type: Inventory (ID: InventoryPolicy_SIM_FULL)
- Table Type: Locations (ID: Location_GFA_NO_SIM_TO_FULL)
- Table Type: Paths (ID: PathData_SIM_FULL)
- Table Type: Periods (ID: TimePeriod_GFA_NO_SIM_FULL)
- Table Type: Products (ID: Product_NO_SIM_FULL)
- Table Type: Sourcing (ID: SourcingData_SIM_TO_FULL)
- Table Type: Suppliers (ID: SupplierData_SIM_FULL)
- Table Type: Vehicle Types (ID: VehicleType_NO_SIM_FULL)

4.2 Native API - Get Target Objects

```
In [57]: # We apply DECISION 0 (Increase Demand).
# The table ID for Demand is 'DemandData_SIM_FULL'.
# Now we query the specific demand records available in this table.

target_table_id = "DemandData_SIM_FULL"

print(f"--- PATH B (STEP 2): FETCHING OBJECTS FROM {target_table_id} ---")
```

```

try:
    print("Querying the specific records inside the selected table...")

    # Retrieve the specific objects (rows/records) inside the Demand table
    demand_objects = api_instance.get_variable_parameter_objects(
        object_type_id=target_table_id,
        scenario_id=the_cloned_scenario.id,
        skip=0,
        take=1000
    )

    print("The response of OpenApiApi->get_variable_parameter_objects:\n")

    # Print the available objects to see what we can modify
    if demand_objects and hasattr(demand_objects, 'list') and demand_objects.list:
        for obj in demand_objects.list:
            print(f" - Record Name: {obj.name} (Object ID: {obj.id})")

            # Automatically select the first demand record to modify in the next step
            target_object_id = demand_objects.list[0].id
            target_object_name = demand_objects.list[0].name
            print(f"\n[SYSTEM] Automatically selected record for variation: {target_

    else:
        print(f"No records found in the {target_table_id} table.")
        target_object_id = None

except Exception as e:
    print("Exception when calling OpenApiApi->get_variable_parameter_objects: %s

```

--- PATH B (STEP 2): FETCHING OBJECTS FROM DemandData_SIM_FULL ---

Querying the specific records inside the selected table...

The response of OpenApiApi->get_variable_parameter_objects:

- Record Name: Demand for customer: Hanover1, for product: MFP (Object ID: 6807)
- Record Name: Demand for customer: Nuremberg1, for product: Monitor (Object ID: 6808)
- Record Name: Demand for customer: Munich1, for product: MFP (Object ID: 6809)
- Record Name: Demand for customer: Poznan1, for product: PC (Object ID: 6810)
- Record Name: Demand for customer: Hamburg1, for product: Monitor (Object ID: 6811)
- Record Name: Demand for customer: Vienna1, for product: PC (Object ID: 6812)

[SYSTEM] Automatically selected record for variation: Demand for customer: Hanover1, for product: MFP (ID: 6807)

4.3. Native API - Get Parameters

In [58]:

```

# We know the table (Demand) and the specific row (Hanover1).
# Now we need to ask the server what columns (parameters) inside this row can be

print(f"--- PATH B (STEP 3): FETCHING PARAMETERS FOR OBJECT {target_object_id} -

try:
    print("Querying available parameters for the selected demand record...")

    # Retrieve the parameters (columns) available for the specific object

```

```

object_parameters = api_instance.get_variable_parameter_parameters(
    object_id=str(target_object_id),
    object_type_id=target_table_id,
    scenario_id=the_cloned_scenario.id
)

print("The response of OpenApiApi->get_variable_parameter_parameters:\n")

if object_parameters and hasattr(object_parameters, 'list') and object_param
    for param in object_parameters.list:
        print(f" - Parameter Name: {param.name} (Parameter ID: {param.id})"

        # Automatically select the first available parameter to proceed
        target_parameter_id = object_parameters.list[0].id
        target_parameter_name = object_parameters.list[0].name

        print(f"\n[SYSTEM] Automatically selected parameter for variation: {targ
else:
    print("No parameters found for the selected object.")
    target_parameter_id = None

except Exception as e:
    print("Exception when calling OpenApiApi->get_variable_parameter_parameters:

```

--- PATH B (STEP 3): FETCHING PARAMETERS FOR OBJECT 6807 ---
 Querying available parameters for the selected demand record...
 The response of OpenApiApi->get_variable_parameter_parameters:

- Parameter Name: Order interval, days (Parameter ID: period)
- Parameter Name: Quantity (Parameter ID: quantity)
- Parameter Name: Revenue (Parameter ID: revenue)
- Parameter Name: Minimum Split Ratio (Parameter ID: minSplitRatio)
- Parameter Name: Expected lead time (Parameter ID: expectedLeadTime)

[SYSTEM] Automatically selected parameter for variation: Order interval, days (ID: period)

4.4. Native API - Get Variation Types

```

In [59]: # We know the table (Demand), the row (Hanover1), and the column (Quantity).
        # Now we ask the server what mathematical operations we can apply.

        # We manually override the target parameter to 'quantity' based on DECISION 0
        target_parameter_id = "quantity"

        print(f"--- PATH B (STEP 4): FETCHING VARIATION TYPES FOR {target_parameter_id}.u

        try:
            print(f"Querying available operations for parameter '{target_parameter_id}'.

            # Retrieve the variation types (math operations) available
            variation_types = api_instance.get_variable_parameter_variation_types(
                object_id=str(target_object_id),
                object_type_id=target_table_id,
                parameter_id=target_parameter_id,
                scenario_id=the_cloned_scenario.id
            )

            print("The response of OpenApiApi->get_variable_parameter_variation_types:\n

```

```

if variation_types and hasattr(variation_types, 'list') and variation_types:
    for v_type in variation_types.list:
        print(f" - Operation: {v_type.name} (Variation Type ID: {v_type.id})

        # For a 20 percent increase, we typically look for a 'Percentage' or 'Mu
        # We temporarily select the first one, but we will adjust based on the o
        target_variation_type_id = variation_types.list[0].id
        target_variation_type_name = variation_types.list[0].name

        print(f"\n[SYSTEM] Automatically selected variation type: {target_variat
    else:
        print("No variation types found for the selected parameter.")
        target_variation_type_id = None

except Exception as e:
    print("Exception when calling OpenApiApi->get_variable_parameter_variation_t

```

--- PATH B (STEP 4): FETCHING VARIATION TYPES FOR QUANTITY ---

Querying available operations for parameter 'quantity'...

The response of OpenApiApi->get_variable_parameter_variation_types:

- Operation: Number Range (Variation Type ID: NUMBER_RANGE)

[SYSTEM] Automatically selected variation type: Number Range (ID: NUMBER_RANGE)

4.5. Native API - Fetch Current Data

```

In [60]: # To apply the 20 percent increase, we need to know the current quantity.
        # We request the variation table data for our selected object.

        print(f"--- PATH B (STEP 5): FETCHING CURRENT DATA FOR {target_object_id} ---")

        try:
            print("Fetching the table data to find the original quantity value...")

            # We use the IDs gathered in the previous blocks
            table_data = api_instance.get_new_variable_parameter_table_data(
                object_id=str(target_object_id),
                object_type_id=target_table_id,
                parameter_id=target_parameter_id,
                rc_id=cloned_RC.id,
                skip=0,
                take=10,
                variation_type=target_variation_type_id
            )

            print("The response of OpenApiApi->get_new_variable_parameter_table_data:\n")
            pprint(table_data)

            print("\n[SYSTEM] Table data fetched successfully. Ready to extract original

        except Exception as e:
            print("Exception when calling OpenApiApi->get_new_variable_parameter_table_d

```

```

--- PATH B (STEP 5): FETCHING CURRENT DATA FOR 6807 ---
Fetching the table data to find the original quantity value...
Exception when calling OpenApiApi->get_new_variable_parameter_table_data: (500)
Reason:
HTTP response headers: HTTPHeaderDict({'Server': 'nginx', 'Date': 'Sat, 18 Apr 20
26 09:25:30 GMT', 'Content-Type': 'application/json', 'Transfer-Encoding': 'chunk
ed', 'Connection': 'keep-alive', 'Vary': 'origin,access-control-request-method,ac
cess-control-request-headers,accept-encoding', 'X-Content-Type-Options': 'nosnif
f', 'X-XSS-Protection': '1; mode=block', 'Cache-Control': 'no-cache, no-store, ma
x-age=0, must-revalidate', 'Pragma': 'no-cache', 'Expires': '0', 'X-Frame-Option
s': 'DENY', 'Strict-Transport-Security': 'max-age=31536000'})
HTTP response body: {"message":"Tables exist only for link data types","details":
{}}

```

4.5. Native API - Fetch Current Data (ALTERNATIVE): Apply Variation without fetching data

```

In [63]: # CREATE VARIATION IN THE CORRECT EXPERIMENT TYPE
# We must use the 'VARIATION' run configuration ID, not the 'SIMULATION' one,
# because variable parameters can only be attached to Variation experiments.

print(f"--- PATH B (FINAL STEP): APPLYING 20% INCREASE TO {target_object_id} ---")

try:
    print("Fetching the VARIATION experiment configuration instead of SIMULATION")

    # 1. Fetch the run configurations again and select the 'VARIATION' type
    cloned_run_configurations = api_instance.get_experiments(the_project.id, the
    variation_RC = next((r for r in cloned_run_configurations if r.type == 'VARI

    if not variation_RC:
        print("Error: No VARIATION experiment found for this scenario.")
    else:
        print(f"Found VARIATION experiment (ID: {variation_RC.id}). Configuring

        # 2. We construct the ID objects needed by the API based on previous blo
        object_type_info = openapi_client.ApiIdNameObject(id=target_table_id)
        object_info = openapi_client.ApiIdNameObject(id=str(target_object_id))
        parameter_info = openapi_client.ApiIdNameObject(id=target_parameter_id)
        variation_type_info = openapi_client.ApiIdNameObject(id=target_variation

        # 3. We define the actual mathematical variation (20% increase)
        variation_data = openapi_client.ApiVariationDataObject(
            body={
                "min": 1.20,
                "max": 1.20,
                "step": 0.0
            }
        )

        # Assemble the main variation object
        api_variation = openapi_client.ApiVariation(
            object_type=object_type_info,
            object=object_info,
            parameter=parameter_info,
            variation_type=variation_type_info,
            data=variation_data

```

```

)

# Assemble the full request body USING THE VARIATION RC ID
variation_request = openapi_client.ApiVariationCreateRequest(
    rc_id=variation_RC.id,
    api_variation=api_variation
)

print("Sending the variation command to the server...")

# Send the request to create the parameter
created_variation = api_instance.create_new_variable_parameter(body=vari

print("The response of OpenApiApi->create_new_variable_parameter:\n")
pprint(created_variation)

print("\n[SYSTEM] DECISION 0 (20% Demand Increase) successfully applied

# Save this specific RC ID for the final step (running the experiment)
target_experiment_rc_id = variation_RC.id

except Exception as e:
    print("Exception when calling OpenApiApi->create_new_variable_parameter: %s\

```

```

--- PATH B (FINAL STEP): APPLYING 20% INCREASE TO 6807 ---
Fetching the VARIATION experiment configuration instead of SIMULATION...
Found VARIATION experiment (ID: 192879). Configuring request...
Sending the variation command to the server...
The response of OpenApiApi->create_new_variable_parameter:

```

```

ApiVariation(data=ApiVariationDataObject(body={'min': 1.2, 'max': 1.2, 'step': 0.
0}), id=194365, object=ApiIdNameObject(id='6807', name='i18n-ValidationDemand_toS
tring i18n-ValidationDemand_ForCustomer: Hanover1, i18n-Validation_ForProduct: MF
P'), object_type=ApiIdNameObject(id='DemandData_SIM_FULL', name='i18n-TableManage
r_DemandTableName'), parameter=ApiIdNameObject(id='quantity', name='i18n-Paramete
r-Quantity'), variation_type=ApiIdNameObject(id='NUMBER_RANGE', name='i18n-Variat
ion_NumberRange'))

```

```

[SYSTEM] DECISION 0 (20% Demand Increase) successfully applied to the VARIATION e
xperiment!

```

=====

DEVELOPMENT TESTS - IGNORE THEM

◀ ————— ▶

```

In [ ]: #Avoid self-signed warnings
import os
import urllib3

```

For self-signed certificates it is necessary to download them and add to the certification path

```

In [ ]: import ssl
import socket

def download_self_signed_cert_no_validation(hostname, port, cert_file):

```

```

context = ssl._create_unverified_context()
conn = context.wrap_socket(socket.socket(socket.AF_INET), server_hostname=hostname)
conn.connect((hostname, port))
der_cert = conn.getpeercert(binary_form=True)
pem_cert = ssl.DER_cert_to_PEM_cert(der_cert)

with open(cert_file, 'w') as cert_file:
    cert_file.write(pem_cert)

print(f"Certificate saved to {cert_file.name}")

# Example usage
download_self_signed_cert_no_validation(SERVER_IP, 443, MY_CERT_FILENAME)

```

```
In [ ]: # !/usr/bin/openssl s_client -showcerts -connect {SERVER_IP}:443 </dev/null 2>/dev/null
```

```
In [ ]: MY_CERT_FILE=os.path.abspath(MY_CERT_FILENAME)
if os.path.isfile(MY_CERT_FILE):
    print(f"Cert file OK: {MY_CERT_FILE} ")
else:
    print(f"Cert file ERROR: {MY_CERT_FILE} ")
MY_CERT_FILE
```

```
In [ ]: os.environ['REQUESTS_CA_BUNDLE'] = MY_CERT_FILE

# Create a PoolManager with a custom CA bundle
http = urllib3.PoolManager(
    cert_reqs='CERT_NONE',          # Enforce cert verification
    ca_certs=MY_CERT_FILE           # Path to your self-signed cert
)

urllib3.disable_warnings(urllib3.exceptions.InsecureRequestWarning)
http.request("GET", TEST_URL)
```

```
In [ ]: from urllib3 import HTTPSConnectionPool

pool = HTTPSConnectionPool(
    SERVER_IP,
    port=443,
    cert_reqs='CERT_REQUIRED',
    ca_certs=MY_CERT_FILE
)
```

```
In [ ]: import requests

# Any request you make now will use your certificate
response = requests.get(TEST_URL, verify=False)
print(response.text)
```

```
In [ ]:
```